

Bug Review

1. 数量变化

阶段	保留数量	本阶段排除	说明
Reasoner 判定 MISMATCH	312	—	原始候选全集
Validatorconfirmed	262	50	262 confirmed, 50 not_confirmed
四类标签/人工分类	142	170	删除推理误判 76、SPEC 错误 94；保留 契约不确定 91、可能有 Bug 51
首轮 review 筛选	62	80	删除“大概率不是 Bug”38 条、“影响较小” 42 条
全部逐条人工 Review	19 (21)	43	对剩余 62 条完成复核；最终保留 18 条 实现缺陷和 1 条待确认项。另有 2 条并非 真实 Bug

整体路径可以概括为：

```

312 MISMATCH
├─ Validator : 262 confirmed / 50 not_confirmed
├─ 四类人工审计 :
│   ├── 推理误判 : 76                                删除
│   ├── SPEC 错误 : 94                                删除
│   ├── 契约不确定 : 91 ┌─
│   └─ 可能有 Bug : 51 └─ 142 条进入主清单
│                           │
│                           ├── 大概率不是 Bug : 38    删除
│                           ├── 影响较小 : 42          删除
│                           └─ 62 条进入逐条人工 Review
│                               ├── 最终排除 : 43
│                               └─ 当前保留 : 19
│                                   ├── 实现缺陷 : 18
│                                   └─ 待确认 : 1 ( 061 )

```

需要注意，**confirmed** 与四类人工标签是两套交叉口径，不是严格的串行子集：

- 142 条主清单中有 136 条 **confirmed**、6 条 **not_confirmed**。
- 首轮筛选后的 62 条中有 59 条 **confirmed**、3 条 **not_confirmed**。
- 最终保留的 19 条均为 **confirmed**，但 **confirmed** 仍只表示 probe 复现了差异，不自动等同于真实产品 Bug；061 因此继续标记为待确认。

截至现在：

- 从 312 条降至 19 条，共排除 293 条；
- 总缩减比例约为 **93.9%**，当前保留率约为 **6.1%**；
- 相比首轮筛选后的 62 条，本次完整人工 Review 又排除 43 条，缩减约 **69.4%**；
- 本报告详细记录 21 条人工结论，其中 19 条保留，076、188 两条明确排除，因此“详细记录 21 条”不等于“最终保留 21 条”。

2. 审核范围与结论口径

本报告使用以下分类：

- **确认实现缺陷**：反例符合真实输入语义，实现会产生错误结果或破坏数据，建议修改。
- **低频实现缺陷**：机制上确实存在，但输入少见、影响范围有限，按优先级处理。
- **待确认**：probe 能证明代码机制，但尚未证明真实依赖、真实数据或产品契约满足触发条件。
- **SPEC/Reasoner 问题**：差异来自自动生成的规约、actual post-condition 或分析范围错误，不应据此修改实现。
- **非真实 Bug / 解析器局限**：行为是简化解析器的已知取舍，规约对该辅助函数提出了不必要或不符合用途的要求。

3. 人工 Review 中遇到的问题

3.1 自动生成的规约可能很奇怪或过强

部分 SPEC 会凭空增加实现和产品并未承诺的要求。例如：

- 强制要求某种具体异常类型，而真实契约可能只要求“失败可被上层识别”。如果实现抛出 **ValueError**，SPEC 却指定必须抛出 **ConfigWizardError**，仅凭异常类型不同不能直接认定为产品 Bug。

- 对辅助解析函数要求处理超出设计范围的语法。例如 076 要求保留未配对的 `<`，但该函数只是正则提取器中的模板参数清理工具，并不是完整的 C++ 语法解析器。
- 对失败路径建模不完整。238 的原 SPEC 只争论“是否必须生成计划”，没有要求调用链区分“不需要更新”和“LLM 生成失败”；人工审核需要继续追踪失败状态是否会被上层静默吞掉。

因此，审核时必须先回答“这个要求是否有独立的产品文档、调用方或测试支持”，再判断实现是否违约。

3.2 AI 给出的 Bug 分类只能参考

可能有 Bug、契约不确定，甚至 Validator 的 `confirmed` 都不能作为最终结论。`confirmed` 通常只说明 probe 成功制造了“实现与生成 SPEC 不一致”，不能证明：

1. SPEC 本身正确；
2. 反例属于真实输入域；
3. 触发条件能在生产调用链出现；
4. 差异会产生实际影响。

让ai初步分了四类，等到人工审核的时候发现依然存在spec过强的情况，但ai第一次认定为可能存在的缺陷，等到第二次review才发现是spec的问题。

3.3 Probe 也可能只验证了人为设定的前提

Mock 返回值、手工构造 SQLite 数据库或语法上极少见的字符串，适合证明某段分支逻辑存在，但不一定能证明真实系统会进入该分支。

函数安全性，只会管理本函数的安全，但有时的输入校验在调用函数前一步已经完成，输入一定是合法的，这个是无法在单函数验证得到的，进而产生误报。也就是说函数的安全性漏洞不会出现在调用链中。

4. 审核结论汇总

ID	简述	人工分类	建议
014	空 TOML 文件被当成文件不存在	确认实现缺陷	修改并补回归测试
031	URL 非数字端口未校验	确认实现缺陷	增加 authority/port 校验

ID	简述	人工分类	建议
061	跨语言调用边被忽略	待确认	用真实 CodeGraph 项目验证
157	域名匹配大小写敏感	确认实现缺陷	规范化 hostname 后比较
*188	<code>main()</code> 被错误分析为只有前 40 行	SPEC/Reasoner 问题	修复分析产物，不改业务实现
238	LLM 生成失败与正常跳过共用 <code>None</code> ，可能静默漏验	确认实现缺陷	返回结构化状态；缺失新规约时阻止成功收尾
239	LLM 返回不完整 <code>new_info</code> 时覆盖并丢数据	确认实现缺陷	合并或完整性校验后原子写入
030	quoted TOML key 使正则匹配失败	确认实现缺陷	支持 quoted key 或使用 TOML-aware 编辑
038	子进程非零退出仍报告成功	确认实现缺陷	检查 <code>returncode</code> 或使用 <code>check=True</code>
071	模板参数中的 <code>operator</code> 被误认成重载声明	低频实现缺陷	先清理模板区再匹配；补真实语法用例
073	Python 多行签名可能被截断	低频实现缺陷	使用 <code>ast/tokenize</code> 或完善续行状态
076	未配对 <code><</code> 被删除	非真实 Bug / 解析器局限	不修改；必要时记录限制
097	已含 <code>server</code> 的 ELP 命令被重复追加	低影响健壮性缺陷	末尾不是 <code>server</code> 时才追加
107	Erlang 分析异常被吞并为空调用图	确认实现缺陷	区分分析失败与真实空图，保留降级或报错信号
139	增量 <code>phases.json</code> 重写后可能仍覆盖不完整	确认实现缺陷	重写后仍必须检查当前源码覆盖率
177	OpenCode 意外异常被记录为成功	低频实现缺陷	异常路径设置失败状态，成功条件同时检查 <code>error</code>
198	非 UTF-8 JSON 触发未捕获解码异常	低频实现缺陷	固定 UTF-8 并捕获 <code>UnicodeDecodeError</code>

ID	简述	人工分类	建议
225	caller 更新未保护原有 callee 条目	确认实现缺陷	由代码合并目标条目并校验无关条目未丢失
252	Bug 目录消失后保留旧 pending 数量	低影响健壮性缺陷	每次扫描开始时重置全部 Bug 计数
272	跨行块注释中的括号污染代码块深度	确认实现缺陷	跨行保存块注释状态后再统计括号
301	嵌套泛型括号导致调用关系漏解析	条件性实现缺陷	fallback 改用平衡括号扫描器或语法树

4.1 真实可触发性复核

总条目21, 225和239合并, 076非bug, 061待确认, 余18条触发路径

不建议作为真实 Bug：076

重复记录：225 和 239 应合并

原 probe 需要修正但问题仍真实：177

依赖 fallback 或特定配置：071、073、097、301

低概率损坏/异常路径：107、177、198

只影响界面：252

待确认项 061

非bug可优化 076

非bug,但反映特殊问题 188

5. 逐条审核记录

FMA-MISMATCH-014：空配置文件被当成不存在

位置： `src/configure_llm.py:1039-1056`, `apply_llm_settings_update`

SPEC: 当 `toml_path` 不存在，或更新结果不是有效 TOML 时抛出 `ConfigWizardError`；否则更新 `[llm]`，保留其他内容，写入前创建备份。

预期行为: 已存在的空文件是合法的空 TOML 文档。函数应调用 `update_llm_settings_toml_text`，生成 `[llm]` 节并写回，而不是报告文件不存在。

实际行为: `_read_text_if_exists` 对空文件返回 `""`，随后 `if not toml_text` 同时覆盖“文件不存在”和“文件存在但内容为空”两种情况：

```
toml_text = _read_text_if_exists(toml_path)
if not toml_text:
    raise ConfigWizardError(
        f"fm-agent.toml not found at {toml_path}; refusing to guess a new
        project config."
    )
```

可能触发理由: 用户预先创建了空的 `fm-agent.toml`，或部署/初始化工具先创建文件再逐步写入配置。空字符串可被 `tomllib.loads("")` 正常解析为 `{}`，下游编辑函数也已经显式支持空文本。

反例:

```
toml_path.write_text("")
apply_llm_settings_update({"backend": "opencode"}, toml_path)
```

预期生成包含 `[llm]` 和 `backend` 的 TOML；实际抛出“not found”。

分类: 确认实现缺陷。 人工结论为“应当修改，应对空文件正常处理”。建议将“是否存在”与“内容是否为空”分开判断，并补空文件回归测试。

FMA-MISMATCH-031: URL 端口未被完整校验

位置: `src/configure_llm.py:90-95`, `validate_base_url`

SPEC: 只接受语法有效、使用 `http` 或 `https` 的绝对 URL；其他输入抛出 `ConfigWizardError`。

预期行为: 除 `scheme` 和 `netloc` 外，还应验证 `authority` 可被正常解释，尤其是端口必须是合法整数且处于有效范围。

实际行为： 当前仅检查 `scheme` 和 `netloc` 是否非空：

```
parsed = urlparse(url)
if parsed.scheme not in ("http", "https") or not parsed.netloc:
    raise ConfigWizardError(...)
```

`urlparse("http://example.com:abc")` 会产生非空 `netloc`，因此函数正常返回；只有进一步访问 `parsed.port` 时 Python 才会因非数字端口抛出 `ValueError`。

可能触发理由： 用户手工配置代理、私有模型服务或复制错误的端口。校验阶段接受后，错误会延迟到 HTTP 客户端初始化或请求阶段，错误位置更难理解。

反例：

```
validate_base_url("http://example.com:abc") # 实际：正常返回
```

预期在配置校验阶段抛出 `ConfigWizardError`。

分类： 确认实现缺陷。这是输入校验遗漏，不只是错误类型争议。建议显式访问并校验 `parsed.hostname`、`parsed.port`，把 `ValueError` 转换为配置错误。

FMA-MISMATCH-061：跨语言调用边被过滤

位置： `src/languages/codegraph.py:382-457`，
`CodeGraphExtractor.get_call_edges`

SPEC： 返回给定语言源文件中发现的所有直接调用关系，调用者和被调用者都用规范化 FQN 表示，并保留 `CodeGraph` 节点 ID 所确定的精确关联。

预期行为： 请求 Python 调用边时，如果 Python 函数直接调用 C/C++ 等其他语言中的函数，结果中仍应保留 `Python caller -> foreign-language callee`。

实际行为： `_node_fqn_map(cur, cg_langs)` 只为当前请求语言建立节点到 FQN 的映射。SQL 查询只限制调用者语言，但随后要求调用者、被调用者都存在于该映射：

```
fqn_of = _node_fqn_map(cur, cg_langs)
...
caller, callee = fqn_of.get(src_id), fqn_of.get(tgt_id)
```

```
if caller and callee:
    result[caller].add(callee)
```

因此目标节点属于其他语言时，`callee` 为 `None`，整条边被静默丢弃。

可能触发理由： JNI、Python C 扩展、FFI、Rust/C 接口或多语言单体仓库可能产生跨语言关系。但是否触发还依赖当前 `CodeGraph` 版本能否解析并落库这种跨语言 `calls` 边。

反例： `probe` 构造了 Python 节点 `src::py_code-py::py_func`、C 节点 `src::c_code-c::c_func` 及其 `calls` 边。调用 `get_call_edges("python")`：

```
预期：{"src::py_code-py::py_func": {"src::c_code-c::c_func"}}
实际：{}
```

分类：待确认。 合成数据库已经证明过滤机制存在，但尚不能证明真实 `CodeGraph` 会生成跨语言调用边，也需确认产品契约是否要求跨语言图。建议用最小真实 FFI 项目执行 `codegraph init` 后检查数据库和函数输出。

FMA-MISMATCH-157：域名匹配错误地依赖大小写

位置： `src/llm_client.py:61-68`, `_matches_inject_target`

SPEC： 无 `scheme` 的 `target` 按 `hostname` 地址空间匹配：`host` 与 `target` 相等，或 `host` 是 `target` 的子域名时返回 `True`。无法解析 `hostname` 时返回 `False`。

预期行为： DNS `hostname` 比较不区分 ASCII 大小写。`example.com`、`EXAMPLE.COM` 以及相应子域名应被视为同一域名空间。

实际行为： 代码直接执行大小写敏感比较：

```
host = urllib.parse.urlparse(url).hostname or ""
return host == target or host.endswith("." + target)
```

可能触发理由： `settings.inject.hosts` 由用户配置，可能包含大写字符；URL 也可能从环境变量或服务配置中保留不同大小写。错误匹配会导致本应注入的稳定 `user ID` 未被注入。

反例：

```
_matches_inject_target("http://example.com/some/path", "EXAMPLE.COM")  
# 预期 True, 实际 False
```

分类：确认实现缺陷。修复时只应对 hostname 分支做标准化，例如比较 `host.lower()` 与 `target.rstrip(".").lower()`；带 scheme 的完整 URL 前缀分支是否大小写敏感应继续按其独立契约处理。

FMA-MISMATCH-188: Reasoner 截断了函数分析

位置： `src/generate_batch_prompts.py:371-508`, `main`

SPEC: dry-run 时打印批处理计划；非 dry-run 时生成 prompt、manifest，清理过期批处理文件；合法输入最终返回 0。

预期行为: `main()` 应执行完整的批处理生成流程。

实际行为: 真实源码确实包含完整流程：遍历 layer 和 batch、调用 `build_prompt`、处理 resume、dry-run 输出、创建输出目录、清理 stale 文件、写入 prompt 和 `manifest.json`，最后返回 0。

原 mismatch 中所谓的“实际行为只执行前 40 行初始化”不是程序运行结果，而是 Reasoner 对抽取函数生成 actual post-condition 时漏读或忽略了后半段。

分类：**SPEC/Reasoner 分析问题**，不是实现 Bug。

但反映出的问题是：因此更可能是 Reasoner 在生成 actual post-condition 时漏读或忽略了后半段，这个问题或许会客观存在。

FMA-MISMATCH-238: LLM 生成失败被折叠为正常跳过，可能造成静默漏验

位置： `src/incremental_reasoner.py:1921-1996`，嵌套函数 `_plan_spec_update`

SPEC: 规约更新阶段必须区分正常跳过与生成失败。文件不存在、语言不支持、已有规约不需要更新、LLM 调用失败和 LLM 返回非法结果不能全部表现为同一个无信息的 **None**；对于没有有效 **sidecar** 的新函数，生成失败必须使流水线进入失败或明确的不完整状态。

预期行为: `_plan_spec_update` 应返回可区分的结构化结果，例如 **updated**、**unchanged**、**missing_file**、**unsupported_language** 和 **generation_failed**。当新函数尚无 `.spec.json/.info.json` 且 LLM 生成失败时，上层应记录该函数失败，阻止流水线以完全成功状态结束，或至少在最终摘要中明确报告未生成、未验证的函数。

实际行为: 函数对多种语义完全不同的情况都返回 **None**：

```
fpath = file_map.get(fqn)
if not fpath or not os.path.isfile(fpath):
    return None

lang_key = EXT_TO_LANG.get(ext)
if not lang_key:
    return None

result = _opencode_generate_spec(...) # 或 _llm_check_spec_update(...)
if not result or not result.get("spec_updated"):
    return None

new_spec = result.get("new_spec")
if not isinstance(new_spec, dict):
    return None
```

底层 LLM/OpenCode 调用可能已经写出错误日志，但 `_plan_spec_update` 没有把失败原因交给上层。并发调度又只保留非空 **plan**：

```
plans[i] = future.result()
...
applied = [p for p in plans if p]
```

因此上层无法知道 **None** 表示“无需更新”，还是某个新函数的规约生成失败。流水线可以继续进入后续阶段；验证逻辑虽然会把新增/修改函数列为目标，但 **reasoner** 会跳过没有有效 **metadata sidecar** 的函数。

可能触发理由: LLM 超时、OpenCode 子进程失败、空响应、JSON 解析失败、响应缺少 **new_spec**，或返回 **spec_updated=false**。对已有且无需修改的规约，**None** 是正常结果；对没有规约的新函数，同样的 **None** 却意味着必要产物没有生成。当前接口无法表达这种差别。

反例： 新增函数 `new_func` 已成功抽取，但尚不存在 `.spec.json` 和 `.info.json`。
`_opencode_generate_spec` 因超时返回 `None`：

1. `_plan_spec_update("new_func", ...)` 返回 `None`；
2. 上层把它从 `applied` 中过滤掉，与“不需要更新”没有区别；
3. 不会为 `new_func` 写入 `sidecar`；
4. 流水线继续运行，验证阶段因缺少有效 `sidecar` 而无法实际验证该函数；
5. 用户如果只看最终完成状态，可能不知道该新函数既没有规约，也没有被验证。

分类：确认实现缺陷。 Bug 不是“LLM 失败时实现必须凭空生成规约”，而是失败状态在层级之间丢失，导致必要产物缺失仍可能被当作正常流程。建议让

`_plan_spec_update` 返回带状态和错误原因的结果；对于无有效 `sidecar` 的新增函数，把 `generation_failed` 汇总为阶段失败或 `incomplete`，并在最终状态、CLI 输出和 Dashboard 中明确列出漏验函数。

FMA-MISMATCH-239：更新 info 时可能覆盖并丢失其他 callee

位置： `src/incremental_reasoner.py:1998-2038`，嵌套函数
`_reconcile_caller`

SPEC： 每个 `changed callee` 依次协调；更新后的整个 `.info.json` 应同时保留此前已经协调的条目和与本次 `callee` 无关的条目。没有修改时文件保持不变。

预期行为： LLM 即使只返回当前 `callee` 的修订，也不能导致其他 `callee` 信息丢失。程序应合并旧条目、验证返回值完整性，或要求并验证 LLM 返回完整 `new_info` 后再写入。

实际行为： 代码把 LLM 返回的 `new_info` 规范化后直接覆盖整个文件：

```
c_new_info = cresult.get("new_info")
if not isinstance(c_new_info, dict):
    continue
with open(f"{cpath}.info.json", "w", encoding="utf-8") as f:
    json.dump(_normalize_info_dict(c_new_info), f, indent=2,
ensure_ascii=False)
```

这里没有检查旧 `c_info["callees"]` 中未涉及的条目是否仍存在。

可能触发理由： LLM 在针对 `callee_func` 做局部协调时，只返回被修改的一项，而省略 `other_func`。这种不完整模型输出属于真实可发生情况；直接覆盖会永久破坏 `sidecar`，进而影响后续 `reasoner` 的调用方契约。

反例：

```
// 写入前
{"callees": [{"name": "callee_func"}, {"name": "other_func"}]}

// LLM 返回
{"callees": [{"name": "callee_func", "post_condition": "UPDATED"}]}

// 实际写入后
{"callees": [{"name": "callee_func", "post_condition": "UPDATED"}]}
```

预期 `other_func` 原样保留，实际被删除。

分类： 确认实现缺陷。 建议按 `callee` 的稳定身份做受控合并，并使用临时文件加原子替换；至少在写入前校验旧的无关 `callee` 集合没有减少。

FMA-MISMATCH-030：正则无法识别 TOML quoted key

位置： `src/configure_llm.py:364-365, 373-435, update_llm_settings_toml_text`

SPEC： 更新 `[llm]` 中指定键，同时保持其他字段、注释和格式；返回值必须仍是有效 TOML。如果 `[llm]` 不存在则追加。

预期行为： TOML 中裸键 `name` 与 quoted key `"name"` 都是合法键写法。更新 `name` 时应识别并替换原行，结果只能保留一个语义上的 `name` 键。

实际行为： `_KV_RE` 只允许 `[A-Za-z0-9_]+` 裸键：

```
_KV_RE = re.compile(r"^(\s*)([A-Za-z0-9_]+)(\s*=\s*)(.*?)(\s*(#.*)?)$")
```

`"name" = "old-model"` 不匹配，旧行被保留；循环结束后又追加裸键 `name = "new-model"`，形成语义重复键和无效 TOML。

可能触发理由： 用户手写 TOML、格式化工具或迁移工具可能输出 quoted key。quoted key 是合法 TOML，不属于越界输入。

反例：

```
[llm]
"name" = "old-model"
provider = "old-provider"
```

执行 `updates={"name": "new-model"}` 后，实际同时包含旧的 `"name"` 和新追加的 `name`；预期只保留值为 `new-model` 的单一键。

分类： 确认实现缺陷。 建议让编辑器识别 TOML quoted key；如果继续使用文本保格式方案，需要正确解析键 token，并在返回前重新 `tomllib.loads` 验证输出。

FMA-MISMATCH-038：子进程非零退出仍报告调用成功

位置： `src/env_check.py:23-32`, `_check_oh_my_openagent`

SPEC： 只有 `bunx oh-my-openagent --version` 在 10 秒内成功响应时返回 `(True, None)`；命令不可用、超时或执行失败时返回 `(False, message)`。

预期行为： 子进程返回码为 0 才表示工具可用；非零返回码必须被视为检查失败。

实际行为：

```
subprocess.run(
    ["bunx", "oh-my-openagent", "--version"],
    capture_output=True, text=True, timeout=10,
)
return True, None
```

没有设置 `check=True`，也没有读取 `CompletedProcess.returncode`。因此正常返回但退出码非零的命令仍被报告为成功；`except` 只覆盖启动异常和超时等抛异常路径。

可能触发理由： `bunx` 本身存在，但包解析失败、包不存在、版本命令内部报错或运行环境缺少依赖。此时 `subprocess.run` 通常正常返回一个非零状态，而不是抛异常。

反例： Mock `subprocess.run` 返回 `returncode=1`、`stderr` 为 `package not found`。实际返回 `(True, None)`；预期返回 `(False, "oh-my-openagent is not installed ...")`。

分类： 确认实现缺陷。 最小修复是保存结果并检查 `returncode`，或使用 `check=True`。消息可以另行改进，不必把所有失败都误报成“未安装”。

FMA-MISMATCH-071：模板区域中的 `operator` 被误认成重载函数

位置： `src/extract.py:195-212`, `_extract_func_name_brace`

SPEC： 对真正的运算符重载声明返回完整 `operator...` 名称；其他声明先移除模板/泛型尖括号区域，再从参数列表前提取普通函数名。

预期行为： `operator...` 只有出现在被解析声明的函数名位置时，才应优先判定为运算符重载；模板参数或其他嵌套文本中的同类片段不应抢占外层函数名。

实际行为： 代码先对未经清理的整个签名执行 `re.search(op_pattern, signature_text)`，命中后立即返回；只有未命中时才调用 `_strip_angle_brackets`：

```
m = re.search(operator_pattern, signature_text)
if m:
    return m.group(1)

cleaned = _strip_angle_brackets(signature_text)
```

可能触发理由： C++ 模板参数、类型表达式或宏展开文本中含有类似 `operator()` 的序列，同时外层声明是普通函数。简化正则会返回错误名称，可能造成提取文件名、FQN 和调用边失配。

反例： 当前 probe 使用：

```
void bar(Template<operator()(int)> t)
```

预期提取 `bar`，实际提取 `operator()`。该字符串足以证明正则顺序存在误判机制，但还应补一个能由目标编译器接受的真实 C++ 模板声明作为回归用例。

分类：低频实现缺陷。识别逻辑确有问题，但现有反例偏合成。建议先用真实语法样本确认，再调整匹配顺序或直接采用语法树结果。

FMA-MISMATCH-073：Python 多行函数签名被截断

位置： `src/extract.py:531-578`, `_extract_functions_indent`

SPEC： 返回每个顶层函数的完整区间，覆盖装饰器、多行函数头和完整函数体，不包含尾部空行。

预期行为： 只要是合法 Python 多行 `def`，无论续行是否额外缩进，提取范围都应覆盖完整签名及函数体。

实际行为： 扫描器仅把“与 `def` 同缩进、且该行以 `)` 开头”的行当作签名续行：

```
if line_indent == indent and re.match(r'\)\s*(:|->)', l.lstrip()):
    j += 1
    continue
if line_indent <= indent:
    break
```

同缩进但以参数名开头的合法续行会触发 `break`，函数区间被截在 `def` 首行。

可能触发理由： Python 允许括号内隐式续行，语法上不要求续行增加缩进。常见格式化工具通常会缩进参数，所以真实出现概率不高，但手写代码或生成代码可能触发。

反例：

```
def foo(a,
b):
    pass
```

预期范围 `[0, 2]`；实际范围只覆盖第 0 行，最终提取源码缺少 `b):` 和 `pass`。

分类：低频实现缺陷。真实可发生但不常见。优先考虑用 `ast/tokenize` 确定 Python 函数范围；继续堆叠缩进正则容易出现更多边界问题。

FMA-MISMATCH-076：未配对尖括号被清理

位置： `src/extract.py:179-192, _strip_angle_brackets`

SPEC： 删除平衡的最外层 `<...>` 区域及未匹配的 `>`，但要求不属于平衡区域的字符保留，因此单独的 `<` 应保留。

预期行为（按生成 SPEC）： `_strip_angle_brackets("<") == "<"`。

实际行为： 函数遇到 `<` 就增加 `depth`，之后只在 `depth == 0` 时保留普通字符；它不会回溯判断 `<` 最终是否闭合：

```
if ch == '<':
    depth += 1
elif ch == '>':
    if depth > 0:
        depth -= 1
else:
    if depth == 0:
        result.append(ch)
```

因此输入 `<` 返回空字符串；未闭合 `<` 后面的文本也会被清理。

可能触发理由： 只有签名文本本身含未配对 `<` 时出现。这通常意味着源码不完整、宏/运算符使简化规则无法区分，或上游函数范围已经错误。

反例： `_strip_angle_brackets("<")`：SPEC 预期 `"<"`，实际 `""`。

分类：非真实 Bug / 解析器局限。 该函数的用途就是粗略移除模板区域，并非通用平衡分隔符解析器；SPEC 对未配对 `<` 和 `>` 还提出了不对称要求。人工结论为“不需要修复，不能为了该反例无脑保留 `<`”。若未来需要完整 C++ 语义，应替换解析策略，而不是针对单字符打补丁。

FMA-MISMATCH-097：ELP 命令可能重复追加 server

位置： `src/languages/erlang.py:57-62, _elp_argv`

SPEC： 返回可用于启动 ELP JSON-RPC server 的 `argv`，最后一个元素为 `server`，前面的元素构成平台适用的命令调用。

预期行为： 配置为 `elp` 时返回 `["elp", "server"]`；如果用户已经配置 `elp server`，不应再次追加同一子命令。

实际行为：

```
command = settings.erlang.command.strip() or "elp"  
argv = shlex.split(command, posix=os.name != "nt")  
if not argv:  
    argv = ["elp"]  
return [*argv, "server"]
```

无论 `argv` 末尾是否已有 `server` 都会追加。

可能触发理由： 用户把 `settings.erlang.command` 理解为完整启动命令并配置成 `elp server`。结果为 `elp server server`，ELP 可能拒绝启动。若配置字段明确只允许“可执行文件及其全局参数”，则该输入不属于推荐配置，因此影响较低。

反例：

```
settings.erlang.command = "elp server"  
_elp_argv()  
# 预期 ["elp", "server"]  
# 实际 ["elp", "server", "server"]
```

分类：低影响健壮性缺陷。 可以做简易安全性修改：

```
if argv[-1:] != ["server"]:  
    argv.append("server")  
return argv
```

同时建议在配置说明中明确 `command` 是二进制命令还是完整 `server` 命令。上面的修复只解决“末尾已经是 `server`”的常见情况；若允许 `elp server --flag`，还需要定义参数应位于子命令前还是后。

FMA-MISMATCH-107：Erlang 分析异常被吞并为空调用图

位置： `src/languages/erlang.py:687-731`, `_analysis_or_none`、`_analysis_or_empty`、`call_edges`

SPEC： `call_edges` 应返回真实分析得到的 Erlang 调用关系；分析后确实不存在调用时可以返回空字典，但分析失败不能被伪装成“成功分析且没有调用边”。

预期行为： ELP 后端不可用时，应向注册器明确表示该后端未能处理，让上层选择 fallback 或报告降级；内部意外异常则至少应保留失败信号，不能把结果当成权威空图。

实际行为： `_analysis_or_none` 捕获所有 `Exception` 并返回 `None`，随后 `_analysis_or_empty` 又把 `None` 转成空分析：

```
def _analysis_or_none(proj_dir):
    try:
        return _analyze_project(proj_dir)
    except Exception as exc:
        logging.warning(...)
        return None

def _analysis_or_empty(proj_dir):
    analysis = _analysis_or_none(proj_dir)
    return analysis or ErlangAnalysis(functions={}, edges={})

def call_edges(proj_dir):
    return _analysis_or_empty(_callgraph_project_root(proj_dir)).edges
```

因此 `call_edges` 即使分析失败也总会返回一个 `dict`。语言注册器会把它视为该后端已处理，而不会再把失败与真实空图区分开。

可能触发理由： ELP 启动或通信失败、缓存/持久化数据异常、响应解析错误，以及实现中的普通编程异常都可能进入宽泛的 `except Exception`。其中前几类属于外部环境问题，后几类更不应被静默降级为空结果。

反例： 令 `_analyze_project` 抛出 `RuntimeError("ELP response parse failed")`。实际 `call_edges` 返回 `{}`；预期是返回可触发 fallback 的失败标记（如 `None`）或传播带上下文的异常，而不是声称项目没有任何调用边。

分类：确认实现缺陷。 原 probe 的 mock 只能证明吞异常机制，但真实异常路径客观存在。其影响是 Erlang 调用边整体缺失，继而使 top-down 分层、caller 上下文和增量影响传播漏项。建议仅捕获可预期的“后端不可用”异常，并统一保留 degraded/error 状态；`call_edges` 不应把失败转换成权威空图。

FMA-MISMATCH-139：增量 phases.json 可能覆写不完全

位置： `src/pipeline_setup.py:1015-1043`, `_run_generate_phases`

SPEC: 增量生成结束时，`phases.json` 不仅要满足 `schema`，还必须覆盖当前分析范围内的源码；文件被重新写入只表示生成器有动作，不等于内容完整。

预期行为: 对增量运行，应在接受新的 `phases.json` 前调用 `_phases_cover_current_sources`。`mtime` 可以用来判断文件是否更新，但不能替代覆盖率校验。

实际行为: 非 `submodule` 的增量分支使用逻辑或：

```
phase_plan_ready = (  
    os.path.getmtime(phases_json) != prev_mtime  
    or _phases_cover_current_sources(phases_json, proj_dir)  
)
```

只要 LLM 重写或 `touch` 了文件，即便新的计划漏掉当前源码，`phase_plan_ready` 仍为 `True`，重试循环立即退出。

可能触发理由: 增量提示要求模型修改已有计划。模型可能成功写出 `schema` 合法的 JSON，却漏掉新文件、重命名文件或某个模块；`mtime` 已变化，所以不完整计划会被接受。

反例: 项目当前有 `main.py`、`a.py`、`b.py`，新 `phases.json` 只列出 `main.py`。若文件 `mtime` 已变化，则实际 `ready` 条件为 `True`，尽管 `_phases_cover_current_sources(...)` 为 `False`。预期继续重试或明确失败。

分类: 确认实现缺陷。这会让后续抽取、分层和验证直接遗漏文件，属于结果完整性问题。建议增量结果始终以覆盖率为必要条件；如还需要确认生成器确实更新过文件，可使用 `mtime_changed and coverage_complete`，而不是 `or`。

FMA-MISMATCH-177: OpenCode 意外异常被记录为成功

位置: `src/opencode_trace.py:304` 起，`run_opencode_traced`

SPEC: 追踪事件的 `status`、`exit_code` 和 `error` 应与真实执行结果一致。命令正常完成才可标记 `success`；启动、等待或内部处理抛异常时必须留下失败状态。

预期行为: 任何从 `run_opencode_traced` 向外传播的执行异常，都应在 `finally` 写入失败事件；`success` 至少应同时满足退出码为 0 且没有错误。

实际行为： 函数先以成功值初始化状态，只显式处理

`subprocess.CalledProcessError`，最终状态主要由 `exit_code == 0` 决定。若 `_start_opencode_process` 抛出 `FileNotFoundError/OSError`，或等待、流处理出现其他异常，异常会继续向外传播，但 `finally` 仍可能记录：

```
status = success
exit_code = 0
error = null
```

可能触发理由： 可执行文件在环境检查后被移除、工作目录失效、系统资源不足、管道读写异常或等待辅助函数出现未预料错误。频率不高，但这些恰好是追踪系统应该准确记录的失败。

反例： 将 `_start_opencode_process` 替换为抛出 `FileNotFoundError("opencode")`。函数确实向调用方抛异常，但落盘事件仍显示 `success`、退出码 `0` 且没有错误信息。

分类： 低频实现缺陷。原 `mismatch` 若假设 `_wait_opencode_process` 正常返回 `(0, "mock error")`，该前提不符合该辅助函数的常规返回约定；但启动阶段或其他未捕获异常造成的错误状态记录是真实问题。主要影响可观测性、统计和排障，不会把异常本身隐藏给主流程。建议捕获通用异常以设置 `error`/非零状态后再抛出，并将成功条件改为 `exit_code == 0 and error is None`。

FMA-MISMATCH-198：非 UTF-8 JSON 导致校验器自身抛异常

位置： `src/file_utils.py:160-166`, `_json_file_is_valid`

SPEC： JSON 文件不可读取或内容无效时返回 `False`，有效时返回 `True`；作为完整性检查函数，不应因坏文件中断恢复流程。

预期行为： 非 UTF-8、截断在多字节字符中间或以其他编码写入的 `.json`，都应被判定为无效 JSON 并返回 `False`。

实际行为： 代码只捕获文件系统错误和 JSON 语法错误：

```
try:
    with open(path, "r") as f:
        json.load(f)
    return True
```

```
except (OSError, json.JSONDecodeError):  
    return False
```

文本解码发生在 `json.load` 解析前；遇到非法编码字节时抛出 `UnicodeDecodeError`，不属于现有捕获范围，因此异常外泄。

可能触发理由： 进程崩溃导致 UTF-8 多字节字符只写入一部分、文件损坏、人工用 GBK 等编码编辑，或旧工具生成了非 UTF-8 JSON。正常由本项目原子写入的 JSON 很少触发。

反例： 创建内容为 `b'\xff\xfe{'` 的 JSON 文件后调用 `_json_file_is_valid(path)`。预期返回 `False`；实际抛出 `UnicodeDecodeError`，可能使 incomplete verification 或 domain context 扫描中断。

分类： 低频实现缺陷。机制真实，但主要面向损坏/外部修改文件。建议显式以 `encoding="utf-8"` 审核报告因此同时记录“反例能证明什么”和“仍需确认什么”。打开，并捕获 `UnicodeDecodeError`；也可统一捕获 `UnicodeError`，让坏文件进入已有的重新生成路径。

FMA-MISMATCH-225: caller 更新未保护原有 callee 条目

位置： `src/incremental_reasoner.py:1204、1650、1998-2038`, `_validate_caller_info_update`、`_llm_check_caller_info_update`、`_reconcile_caller`

SPEC： 针对某个 changed callee 更新 caller 的 `.info.json` 时，只能修改与该 callee 有关的信息；原文件中其他 callee 条目必须保留。

预期行为： “保留非目标 callee”应由程序确定性保证。可以让 LLM 只返回目标条目并由代码合并，也可以要求完整返回，但写入前必须比较并拒绝丢失无关条目的结果。

实际行为： 提示词虽然要求模型保留其他信息，结构校验器却只检查响应 schema。`_reconcile_caller` 随后把 `new_info` 当作完整文件直接覆盖：

```
c_new_info = cresult.get("new_info")  
if not isinstance(c_new_info, dict):  
    continue  
with open(f"{cpath}.info.json", "w", encoding="utf-8") as f:
```

```
json.dump(_normalize_info_dict(c_new_info), f, indent=2,  
ensure_ascii=False)
```

没有校验旧 `callees` 中与本次目标无关的条目是否仍在。

可能触发理由： caller 的 callee 较多、上下文较长，或模型把“更新某个 callee”理解成只返回该 callee 的新对象。该响应可能完全符合当前 schema，因此顺利通过验证。

反例： 原文件含 `callee_a` 与 `callee_b`，本轮只协调 `callee_a`；模型返回仅含更新后 `callee_a` 的 `new_info`。实际覆盖后 `callee_b` 消失。预期 `callee_b` 原样保留。

分类：确认实现缺陷。 这是对不可靠模型输出缺少数据保护，与 239 的覆盖问题高度相关：225 强调生成/校验边界没有落实“只改目标 callee”，239 强调最终覆盖写入会造成数据丢失。建议由代码按稳定标识合并目标条目，并校验所有非目标条目等价保留后再原子替换文件。

FMA-MISMATCH-252: Bug 目录消失后 pending 计数遗留

位置： `dashboard.py:472` 起，`State.scan_bugs`

SPEC： 每次扫描都应让 `bugs_confirmed`、`bugs_not_confirmed` 和 `bugs_pending` 反映当前磁盘状态；目录不存在时三者均应为 0。

预期行为： 扫描开始时重置全部计数。若 bug validation 目录已经删除，函数返回后不应保留上一次扫描的 pending 数。

实际行为： 函数先重置 confirmed 和 not-confirmed，遇到目录不存在便提前返回；`bugs_pending` 只在目录存在的后续路径中赋值。因此旧 pending 值会留在内存中。

可能触发理由： Dashboard 持续运行时，上一轮扫描发现若干待验证 Bug；新一轮或清理流程删除整个 bug 目录，下一次扫描走提前返回。增量 reasoner 确实包含清理 validation 目录的生命周期。

反例： 第一次扫描存在目录并得到 `bugs_pending = 3`，随后删除该目录并再次调用 `scan_bugs()`。预期 pending 为 0；实际仍为 3，而其他两个计数已重置。

分类：低影响健壮性缺陷。 会造成 Dashboard 总数和状态显示过期，但不影响分析、验证或文件内容。修复只需在存在性检查前同时执行 `self.bugs_pending = 0`，并

FMA-MISMATCH-272：跨行块注释中的括号影响代码块切分

位置： `src/reasoner.py:25` 起，`_compute_brace_depth_per_line`，调用方 `_split_into_blocks_braced`

SPEC： 大括号深度只应由真实代码 token 决定；字符串、行注释和块注释中的 `{}` 不得改变函数结构和安全切分点。

预期行为： 扫描 `/* ... */` 跨行注释时应把 `in_block_comment` 状态传递到下一行，直到遇到结束标记后才恢复括号统计。

实际行为： 逐行计算深度时没有跨行保留块注释状态。若 `/*` 与 `*/` 不在同一行，后续注释行中的大括号会被当成代码括号，污染 `depth`。

可能触发理由： C、C++、Java、JavaScript、TypeScript 等语言常使用跨行块注释；注释中的代码示例、JSON、伪代码或文字大括号很常见。当长函数超过分块粒度时，错误深度会参与选择切分边界。

反例： 在长函数中加入：

```
/* example closes a block:
 * }
 */
if (x) {
    work();
}
```

由于注释中的 `}` 被计数，计算深度可能在真实 `if (x) {` 后错误回到入口深度，使切分器把块截在打开的 `if` 之后。相反，注释中的多余 `{` 也可能令深度一直过高，从而无法在合法位置切分。

分类： 确认实现缺陷。它影响送给 LLM 的代码块完整性，可能导致上下文缺失、误判和额外成本；只在长函数进入 `braced splitter` 时显现。建议使用跨行词法状态机，至少持续跟踪 `block comment`、字符串和转义状态；更稳妥的是使用语言 `parser/tokenizer`。

FMA-MISMATCH-301：嵌套泛型括号导致调用关系漏解析

位置： `src/generate_topdown_layers.py:221-276`, `_get_call_regex`、`_find_call_sites`、`_build_call_graph`

SPEC： fallback 调用图提取应识别合法的泛型/模板函数调用，并把 caller 与 callee 建立关系；泛型参数嵌套不应使外层函数名失配。

预期行为： C++ `foo<A<B>>(x)`、Rust `bar::、Go fn[map[string]int](x) 等合法调用都应至少识别出基础 callee 名。`

实际行为： 调用正则使用类似 `<[^\>]*>` 或方括号的单层模式。`[^\>]*` 在第一个 `>` 停止，无法表达平衡的嵌套泛型；剩余的 `>` 出现在预期调用括号之前，导致整个模式无法匹配。

可能触发理由： 嵌套容器、模板模板参数、Rust turbofish 和 Go 泛型组合在真实代码中都可能出现。不过正常流程会优先使用 CodeGraph；只有 CodeGraph 不可用、初始化失败、语言未注册或某些补充扫描路径进入正则 fallback 时，此问题才主导调用图。

反例： fallback 扫描以下代码：

```
auto value = foo<std::vector<int>>(items);
```

预期 caller 包含到 `foo` 的边；实际正则只消费到 `int>` 的第一个 `>`，无法越过第二个 `>` 匹配 `(`，最终漏掉该调用。

分类： 条件性实现缺陷。解析缺陷真实，影响取决于是否走正则 fallback。漏边会造成 top-down layer 顺序不准、caller 上下文缺失，以及增量影响传播漏掉调用方。正则很难可靠处理任意嵌套，建议改为平衡分隔符扫描器；已支持的语言则优先使用 CodeGraph/AST，并针对 fallback 补嵌套泛型回归测试。

6. 后续维护与复核建议

后续维护报告或复核结论时，建议继续按以下顺序判断：

1. 先核对真实源码是否被完整抽取，避免重复 188 的截断误判。
2. 再确认 SPEC 是否有调用方、文档、schema 或测试作为独立依据。

3. 检查 probe 是否只依赖 mock、非法语法或人工数据库；若是，应补真实调用链验证。
4. 分开记录“代码机制存在”“真实可触发”“会产生实际影响”三个层次。
5. 对写文件、删除数据、调用图丢边和安全匹配类问题提高优先级；对输出格式、异常文案和解析器极端输入降低优先级。